

Récupérer et réutiliser les sorties d'analyse statistique en R

De la console aux objets exploitables

Enjeu

Dans un workflow statistique classique, les résultats sont généralement lus directement dans la console. Cette lecture est essentielle, mais elle reste limitée : la sortie affichée n'est pas encore un matériau facilement réutilisable. L'objectif de cette fiche est de montrer comment transformer une sortie d'analyse en objets R nommés, inspectables et réutilisables.

sortie console $\longrightarrow$ objet R $\longrightarrow$ texte contrôlé $\longrightarrow$ prompt ou interprétation

Un changement de posture

Lire une sortie statistique dans la console répond à une première question :

Que dit le résultat ?

Récupérer cette sortie dans un objet R permet d'aller plus loin :

Comment puis-je réutiliser ce résultat dans une autre étape du workflow ?

Ce changement est central pour construire des analyses reproductibles et des prompts contrôlés.

Deux objets statistiques de départ

Le workflow illustré ici part de deux objets déjà produits par des fonctions de **FactoMineR** :

```
res_lm  
res_aovsum
```

Ces objets contiennent les résultats de deux analyses :

```
FactoMineR::LinearModel()  
FactoMineR::AovSum()
```

Ils ne sont pas seulement des sorties affichées : ce sont des objets R structurés.

Inspecter un objet avec `names()`

Avant d'extraire une information, il faut savoir ce que contient l'objet.

On utilise pour cela :

```
names(res_lm)
names(res_aovsum)
```

La fonction `names()` affiche les composantes accessibles dans l'objet.

Elle répond à la question :

Quelles parties du résultat puis-je récupérer ?

Extraire une composante avec `$`

Un objet R peut contenir plusieurs sous-objets.

On peut récupérer une composante avec l'opérateur `$`.

Exemple :

```
lm_ftest <- res_lm$Ftest
```

Cette instruction signifie :

Je prends la composante `Ftest` de `res_lm` et je la stocke dans un nouvel objet appelé `lm_ftest`.

Sorties extraites de `LinearModel()`

À partir de l'objet `res_lm`, on extrait trois éléments :

```
lm_ftest <- res_lm$Ftest
lm_ttest <- res_lm$Ttest
lm_resume <- res_lm$lmResult
```

Ces objets correspondent à différents niveaux de lecture du modèle linéaire.

Rôle des objets extraits

Objet	Rôle
<code>lm_ftest</code>	Tester les effets globaux du modèle
<code>lm_ttest</code>	Examiner les coefficients du modèle
<code>lm_resume</code>	Conserver le résumé du modèle linéaire

L'intérêt est de ne plus dépendre uniquement de l'affichage console.

Chaque résultat devient un objet nommé.

Sorties extraites de AovSum()

À partir de l'objet `res_aovsum`, on extrait :

```
aov_ftest <- res_aovsum$Ftest  
aov_ttest <- res_aovsum$Ttest
```

Ces deux sorties permettent de distinguer deux niveaux d'interprétation.

Deux niveaux d'interprétation en ANOVA

Objet	Question statistique
<code>aov_ftest</code>	Quels effets globaux sont significatifs ?
<code>aov_ttest</code>	Quels coefficients ou contrastes contribuent à ces effets ?

Cette distinction évite de confondre :

- un effet global ;
- un coefficient particulier ;
- une interaction ;
- une modalité ou un contraste.

Rendre l'affichage plus lisible

Une fois les objets extraits, on peut créer une petite fonction d'affichage.

```
afficher_sortie <- function(titre, commande, objet) {  
  cat("\n===== \n")  
  cat(titre, "\n")  
  cat("Objet créé : ", commande, "\n", sep = " ")  
  cat("===== \n")  
  print(objet)  
}
```

Cette fonction ne modifie pas l'analyse.

Elle rend simplement la sortie plus explicite.

Pourquoi créer une fonction d’affichage ?

La fonction `afficher_sortie()` ajoute trois informations autour de chaque objet :

1. un titre explicite ;
2. la commande qui a créé l’objet ;
3. l’objet imprimé.

Elle permet de relier clairement :

commande R $\rightarrow$ objet créé $\rightarrow$ résultat affiché

Exemple d’utilisation

```
afficher_sortie(  
  titre = "AovSum : effets principaux et interaction - Ftest",  
  commande = "aov_ftest <- res_aovsum$Ftest",  
  objet = aov_ftest  
)
```

Cette instruction rappelle ce qui est affiché, d’où vient l’objet et comment il a été créé.

Elle rend la console plus pédagogique et plus facile à relire.

Capturer une sortie complète sous forme de texte

Pour alimenter un prompt ou conserver une trace textuelle complète, on peut capturer l’affichage d’un objet.

```
texte_linearmodel <- paste(  
  capture.output(print(res_lm)),  
  collapse = "\n"  
)  
  
texte_aovsum <- paste(  
  capture.output(print(res_aovsum)),  
  collapse = "\n"  
)
```

On obtient ici deux objets textuels.

Que fait `capture.output()` ?

La fonction `capture.output()` récupère ce qui aurait été imprimé dans la console.

Autrement dit, elle transforme une sortie affichée en texte.

Exemple :

```
capture.output(print(res_aovsum))
```

Cette instruction capture la sortie complète de `res_aovsum` sans se contenter de la lire à l'écran.

Pourquoi utiliser `paste(..., collapse = "\n")` ?

`capture.output()` renvoie généralement plusieurs lignes de texte.

La fonction :

```
paste(..., collapse = "\n")
```

rassemble ces lignes dans une seule chaîne de caractères, tout en conservant les retours à la ligne.

Le résultat devient un texte structuré, prêt à être réutilisé.

Les objets textuels produits

Objet	Contenu
<code>texte_linearmodel</code>	Sortie complète de <code>LinearModel()</code> transformée en texte
<code>texte_aovsum</code>	Sortie complète de <code>AovSum()</code> transformée en texte

Ces objets ne sont pas seulement des résultats.

Ils deviennent des matériaux pour une étape ultérieure : prompt, rapport, interprétation ou documentation.

Une sortie devient un matériau de prompt

Après extraction et capture, on dispose de deux types d'objets.

Des objets structurés :

```
lm_ftest  
lm_ttest  
lm_resume  
aov_ftest  
aov_ttest
```

Des objets textuels :

```
texte_linearmodel  
texte_aovsum
```

Ces objets peuvent être injectés dans une consigne adressée à un modèle de langage.

Ce que permet cette organisation

Cette organisation rend le workflow plus contrôlable.

On peut :

- choisir précisément les résultats transmis ;
- vérifier les objets avant de les utiliser ;
- éviter de copier-coller manuellement la console ;
- construire des prompts reproductibles ;
- garder une trace inspectable des étapes intermédiaires.

Objets disponibles à la fin du workflow

Objet	Usage possible
lm_ftest	Interpréter les effets globaux du modèle linéaire
lm_ttest	Interpréter les coefficients du modèle linéaire
lm_resume	Conserver le résumé du modèle
aov_ftest	Interpréter les effets principaux et interactions
aov_ttest	Interpréter les coefficients et contrastes
texte_linearmodel	Alimenter un prompt ou un rapport
texte_aovsum	Alimenter un prompt ou un rapport

Point clé

Une interprétation assistée par un modèle de langage ne devrait pas commencer par une demande vague.

Elle devrait commencer par des objets R bien construits.

```
objet statistique  
→ sortie récupérée  
→ texte capturé  
→ prompt contrôlé  
→ interprétation inspectable
```

À retenir

Le workflow présenté ici repose sur une idée simple :

Une sortie statistique devient plus utile lorsqu'elle est récupérée, nommée et transformée en objet réutilisable.

Ce principe permet de passer d'une lecture ponctuelle de la console à une chaîne d'analyse plus robuste, plus transparente et plus reproductible.